BOSTON UNIVERSITY

EC 551 – Advanced Digital Design with Verilog and FPGAs

Lecture 8
Intro to Synthesis

Prof. Douglas Densmore

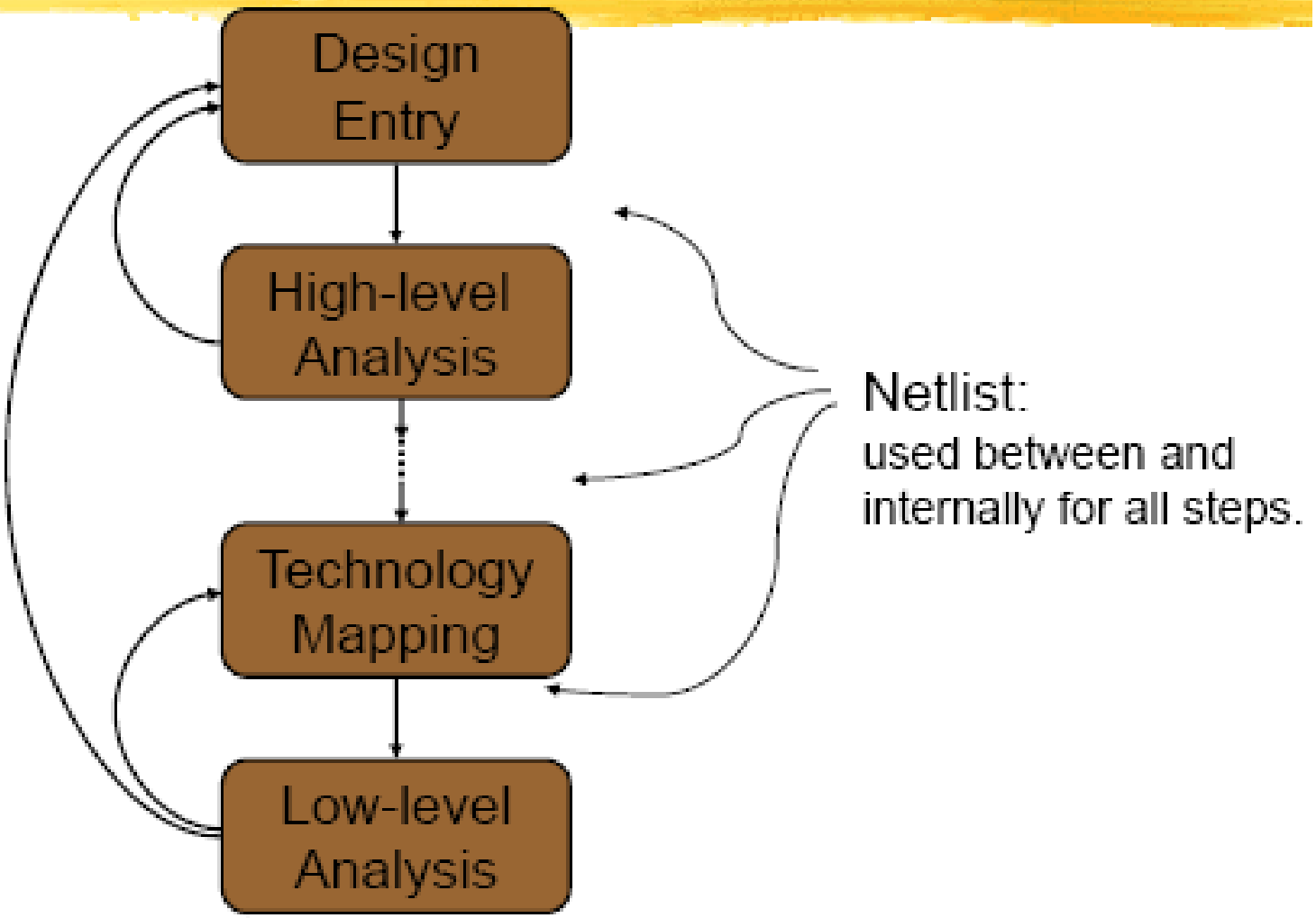
Additional material courtesy of Alexander Taubin, EC551 Fall 2009

Announcements

- Midterm Exam
  - Grading will be slow, hopefully the end of the month
- HW2 due today
- HW3 out; due 11/3
- Program 1 due Friday 10/23
  - Demo slots posted (see Blackboard annoucement)

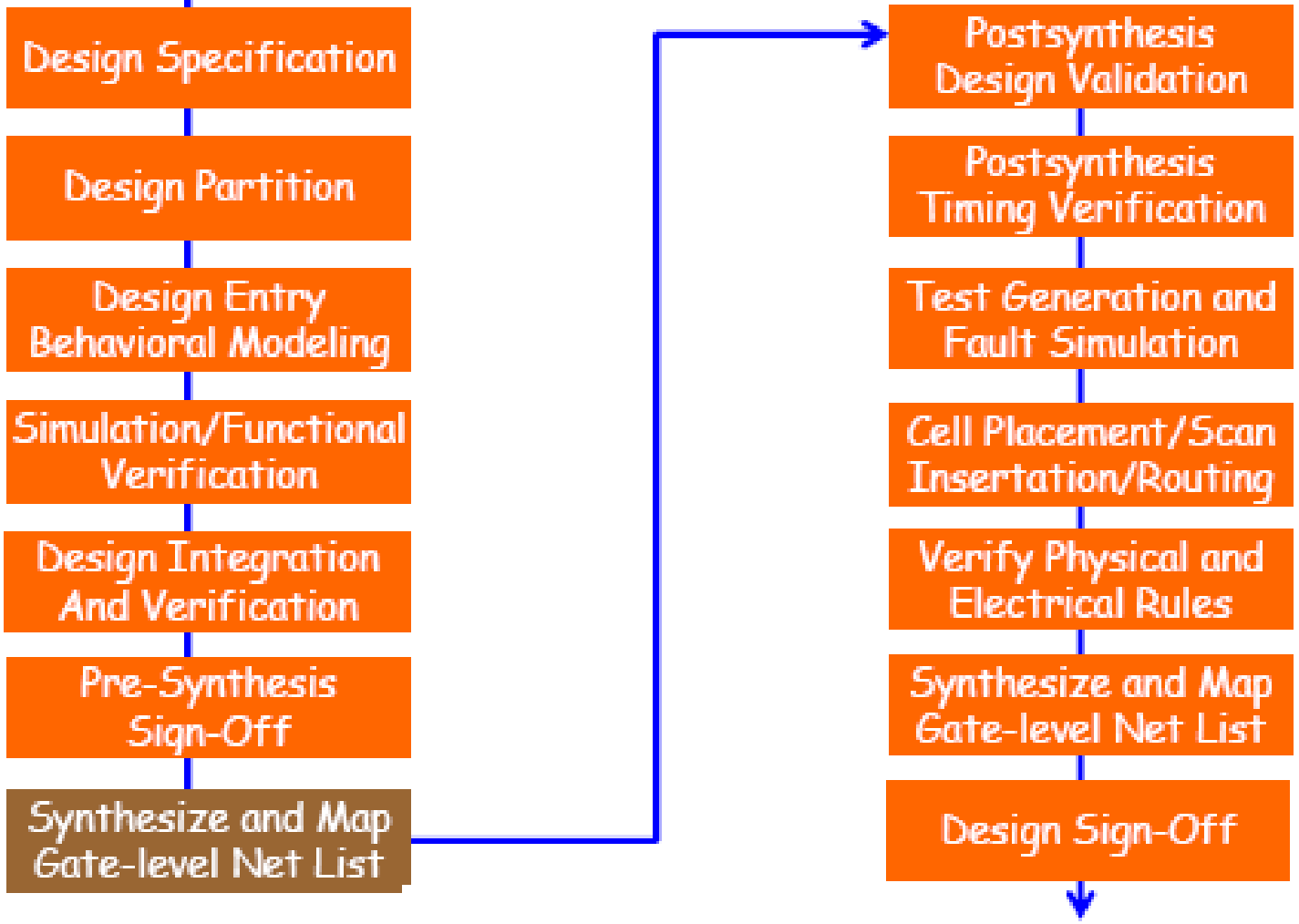
BOSTON UNIVERSITY

Recall: Design Flow



BOSTON UNIVERSITY

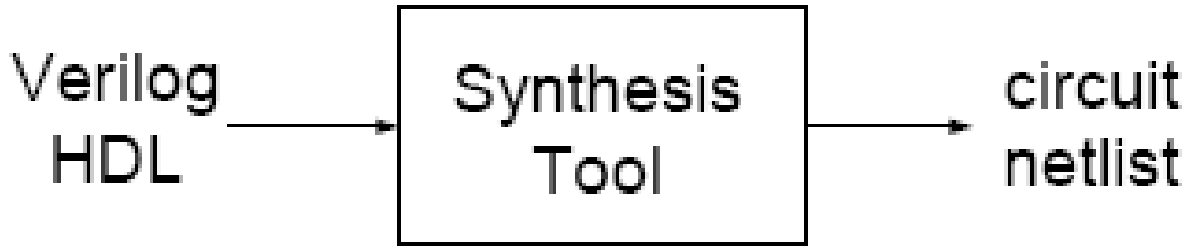
Design Methodology in Detail



BOSTON UNIVERSITY

Logic Synthesis

- Verilog and VHDL started out as simulation languages, but quickly people wrote programs to automatically convert Verilog code into low-level circuit descriptions (netlists).



- Synthesis converts Verilog (or other HDL) descriptions to implementation technology specific primitives:
  - For FPGAs: LUTs, flip-flops, and RAM blocks
  - For ASICs: standard cell gate and flip-flop libraries, and memory blocks.

BOSTON UNIVERSITY

Why Logic Synthesis?

1. Automatically manages many details of the design process:
   - ⇒ Fewer bugs
   - ⇒ Improved productivity
2. Abstracts the design data (HDL description) from any particular implementation technology.
   - Designs can be re-synthesized targeting different chip technologies. Ex: first implement in FPGA then later in ASIC.
3. In some cases, leads to a more optimal design than could be achieved by manual means (ex: logic optimization)

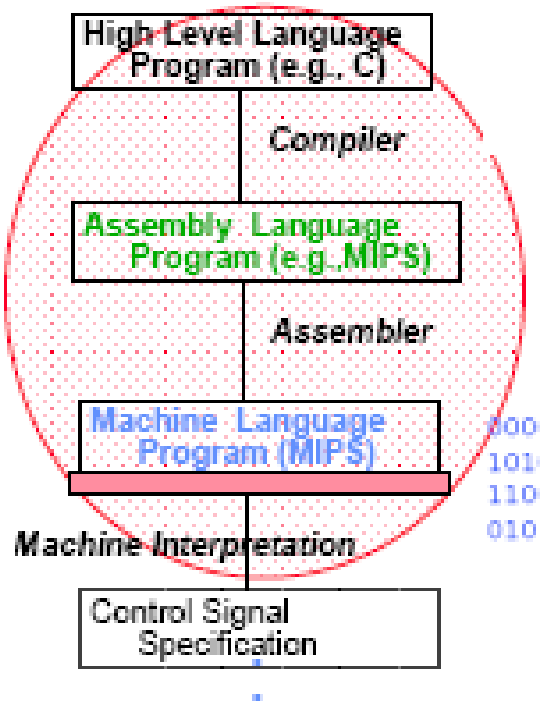
Why Not Logic Synthesis?

1. May lead to non-optimal designs in some cases.

BOSTON UNIVERSITY

Synthesis vs Compilation

Levels of Representation




```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw $to,   0($2)
lw $t1,   4($2)
sw$t1,    0($2)
sw$t0,    4($2)
```

```
0000 1001 1100 0110 1010 1111 01
1010 1111 0101 1000 0000 1001 11
1100 0110 1010 1111 0101 1000 00
0101 1000 0000 1001 1100 0110 10
```

- Compiler
  - recognizes all possible constructs in a formally defined program language
  - translates them to a machine language representation of execution process
- Synthesis
  - Recognizes a target dependent subset of a hardware description language
  - Maps to collection of concrete hardware resources
  - Iterative tool in the design flow

BOSTON UNIVERSITY

Unsupported Language Constructs

Generate error and halt synthesis

- Net types: trireg, wor, trior, wand, triand, tri0, tri1, and charge strength;
- register type: real.
- Built-in unidirectional and bidirectional switches, and pull-up, pull-down.
- Procedural statements: assign (different from the "continuous assignment"), deassign, wait.
- Named events and event triggers.
- UDPs (user defined primitives) and specify blocks.
- force, release, and hierarchical net names (for simulation only).

Simply ignored

- delay, delay control, and drive strength.
- scalared, vectored.
- initial block.
- Compiler directives (except for `define, `ifdef, `else, `endif, `include, and `undef, which are supported).
- Calls to system tasks and system functions (they are only for simulation).

BOSTON UNIVERSITY

How does it work?

- A variety of general and ad-hoc (special case) methods:
  - ***Instantiation:*** maintains a library of primitive modules (AND, OR, etc.) and user defined modules.
  - ***"macro expansion" / substitution:*** a large set of language operators (+, -, Boolean operators, etc.) and constructs (if-else, case) expand into special circuits.
  - ***Inference:*** special patterns are detected in the language description and treated specially (ex: inferring memory blocks from variable declaration and read/write statements, FSM detection and generation from "always @ (posedge clk)" blocks).
  - ***Logic optimization:*** Boolean operations are grouped and optimized with logic minimization techniques.
  - ***Structural reorganization:*** advanced techniques including sharing of operators, and retiming of circuits (moving FFs), and others?

BOSTON UNIVERSITY

Synthesis of Combinational and Sequential Logic

Tasks for synthesis tools:

- detect and eliminate redundant logic
- detect combinational feedback loops
- exploit don't-care conditions
- detect unused states
- detect and collapse equivalent states
- make state assignments
- synthesize optimal, multilevel realizations of logic subject to constraints on area and/or speed physical technology.

BOSTON UNIVERSITY

Tasks for the Designer

- Understand how to synthesize combinational logic
- Understand how to synthesize sequential logic
- Understand how language constructs synthesize (write synthesis-friendly descriptions)
- Anticipate the results of synthesis
- Adhere to style conventions

BOSTON UNIVERSITY

Introduction to Synthesis

Three common levels of abstraction:

- ***architectural***: *sequence of operation*
  - *transform input sequence to output sequence*
  - ***no schedule for clock cycles***
- ***logical***: *variables and Boolean functions*
  - *Fixed architecture of registers, datapaths, and functional units*
  - ***Synthesize an optimized netlist of gates and registers – EC551 focus***
- ***physical***: geometric detail
  - *mask sets for transistor fabrication*

BOSTON UNIVERSITY

Levels of Abstraction

Increasing level of abstraction

Behavior : A model that implements a design **algorithm** in high-level language constructs (for **algorithm & concept exploration**)

RTL : A model that describes the **flow of data** between registers and how a design processes that data (for **synthesis**).

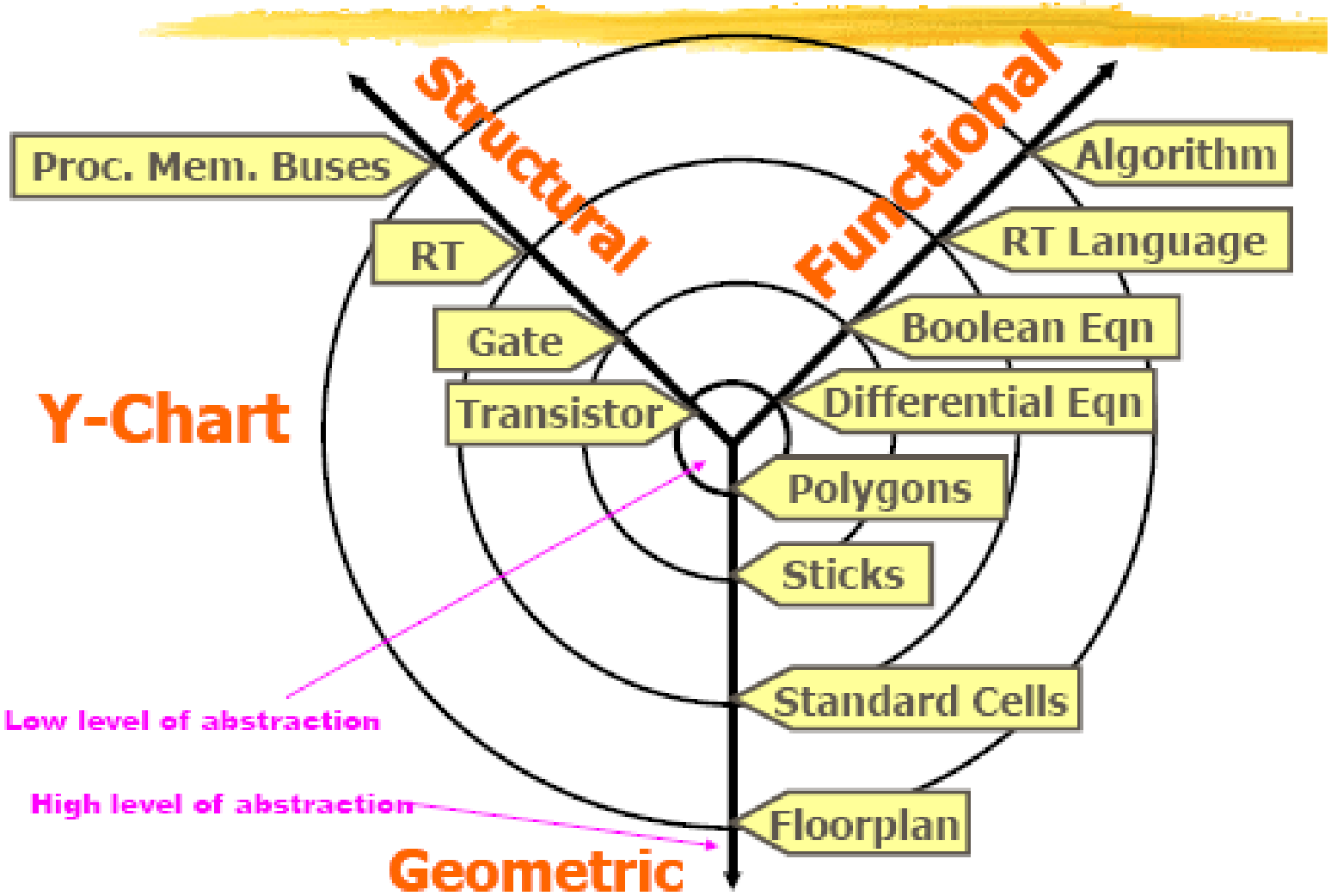
Gate-level : A model that describes the logic gates and the connections between logic gates in a design (for **P&R, functional & timing verification**)

Switch-level : A model that describes the transistors and storage nodes in a device and the connections between them(for **physical verification**).

Geometry : A model that describes different layers for physical implementation (for **manufacturing**).

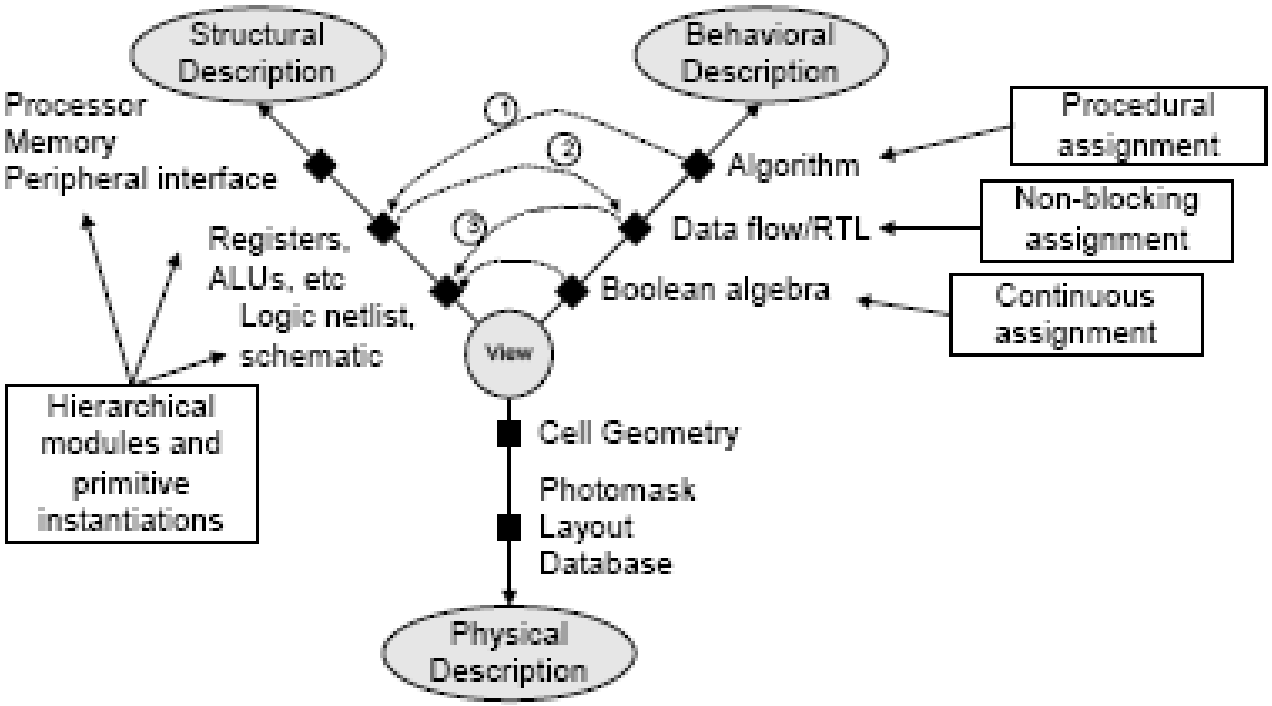
BOSTON UNIVERSITY

HW Design Representation and Abstraction



Modified Y-Chart: Verilog constructs supporting synthesis

- (1) Behavioral synthesis transforms an algorithm to an architecture and a schedule of operations (clock cycles)
- (2) Architecture is represented as an RTL model
- (3) Synthesize RTL model is a netlist of gates and registers



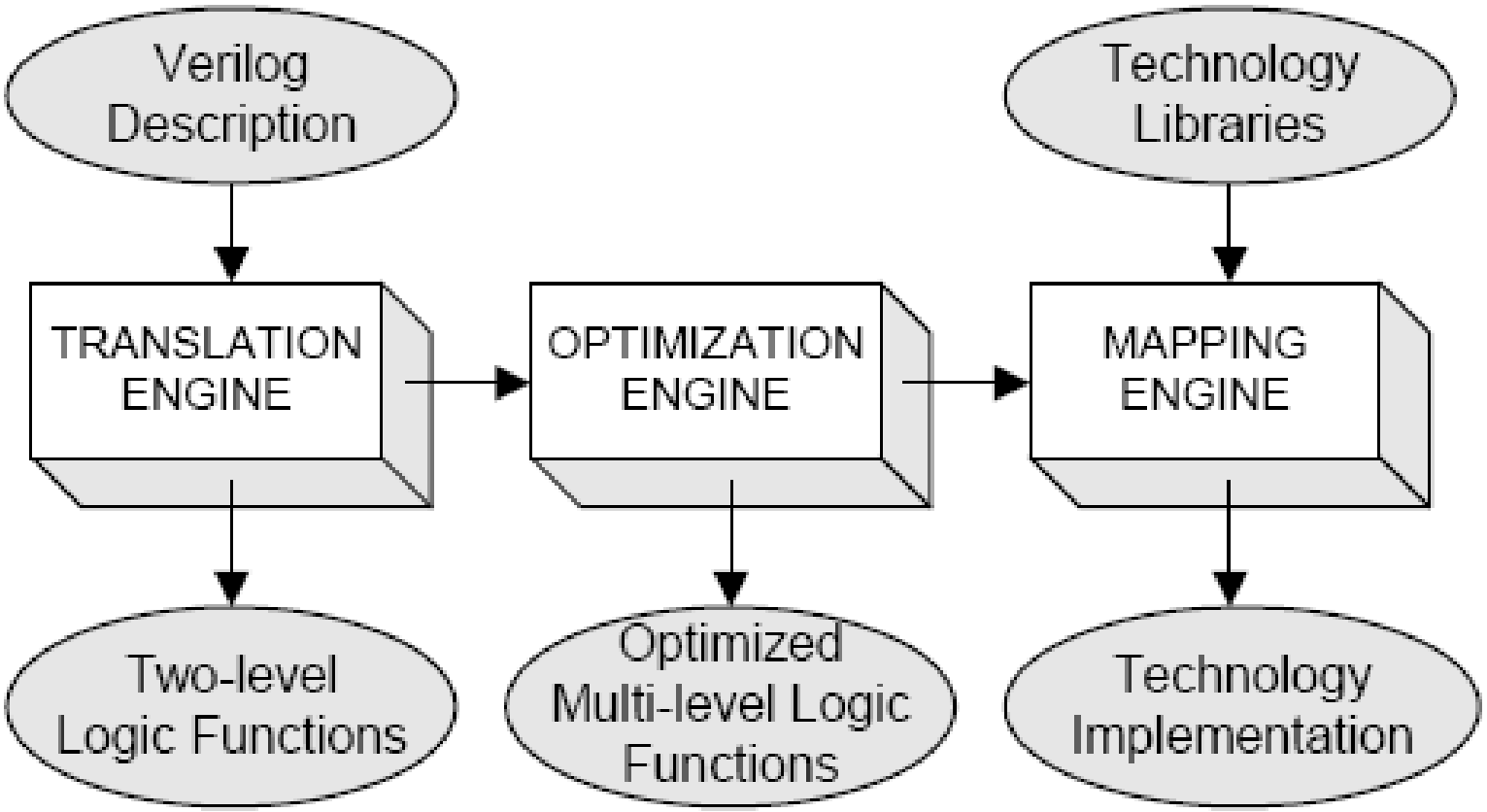
BOSTON UNIVERSITY

Terminologies

- **Resources:** refers to hardware building blocks in the following categories:
  - **Primitives** resources (any circuits with fully characterized area and performance parameters.)
  - **Application** specific resources (any circuits that perform a particular task.)
  - **Memory** resources (registers, read-only, read-write memory blocks.)
  - **Interface** resources (buses and IO pads)
- **Scheduling**: the task of determining the start times for the resources, subject to design constraints.
- **Resource allocation:** refers to the process of **binding** resources to tasks.
- **Chaining:** the process of merging two or more combinational resources in a sequence so that they can be accomplished **within one clock cycle**.
- **Dataflow** : refers to operations and data dependencies.
- **Control Flow**: relates to branching (conditional) and iterations (loops)

BOSTON UNIVERSITY

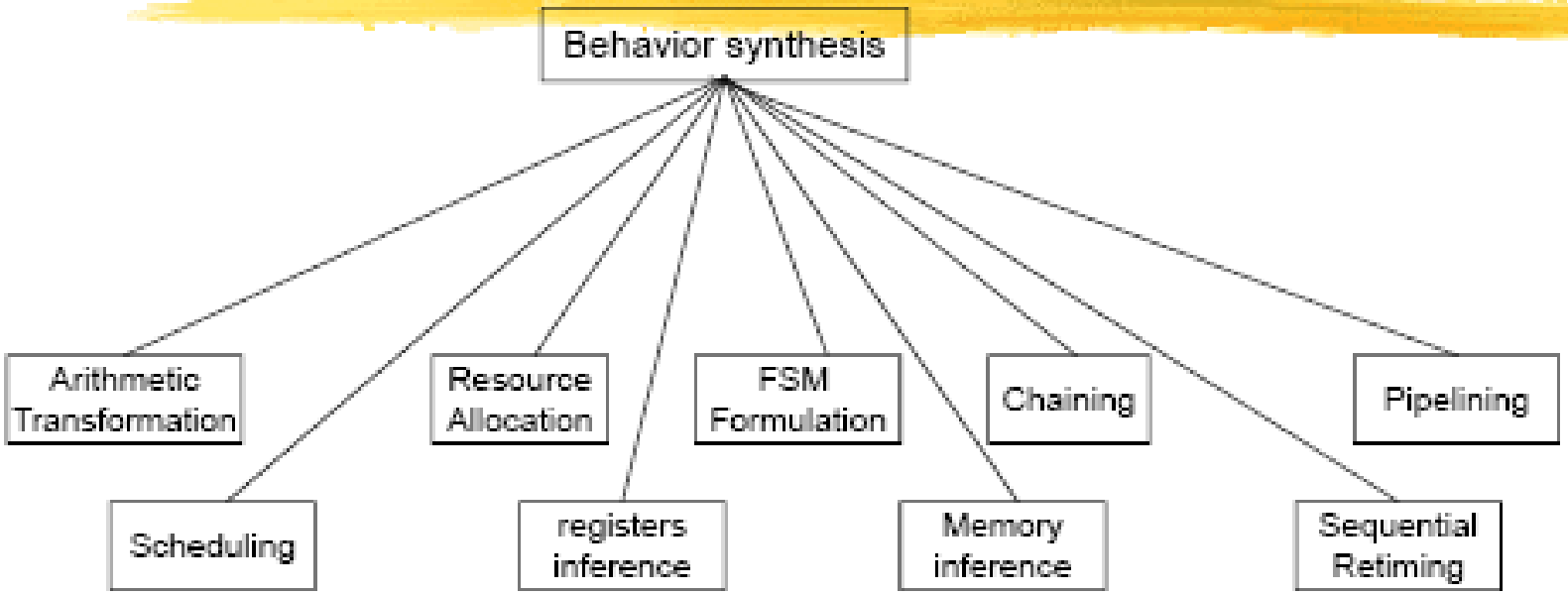
Synthesis Tool Organization



Design Goals: Functionality, area, timing, testability

BOSTON UNIVERSITY

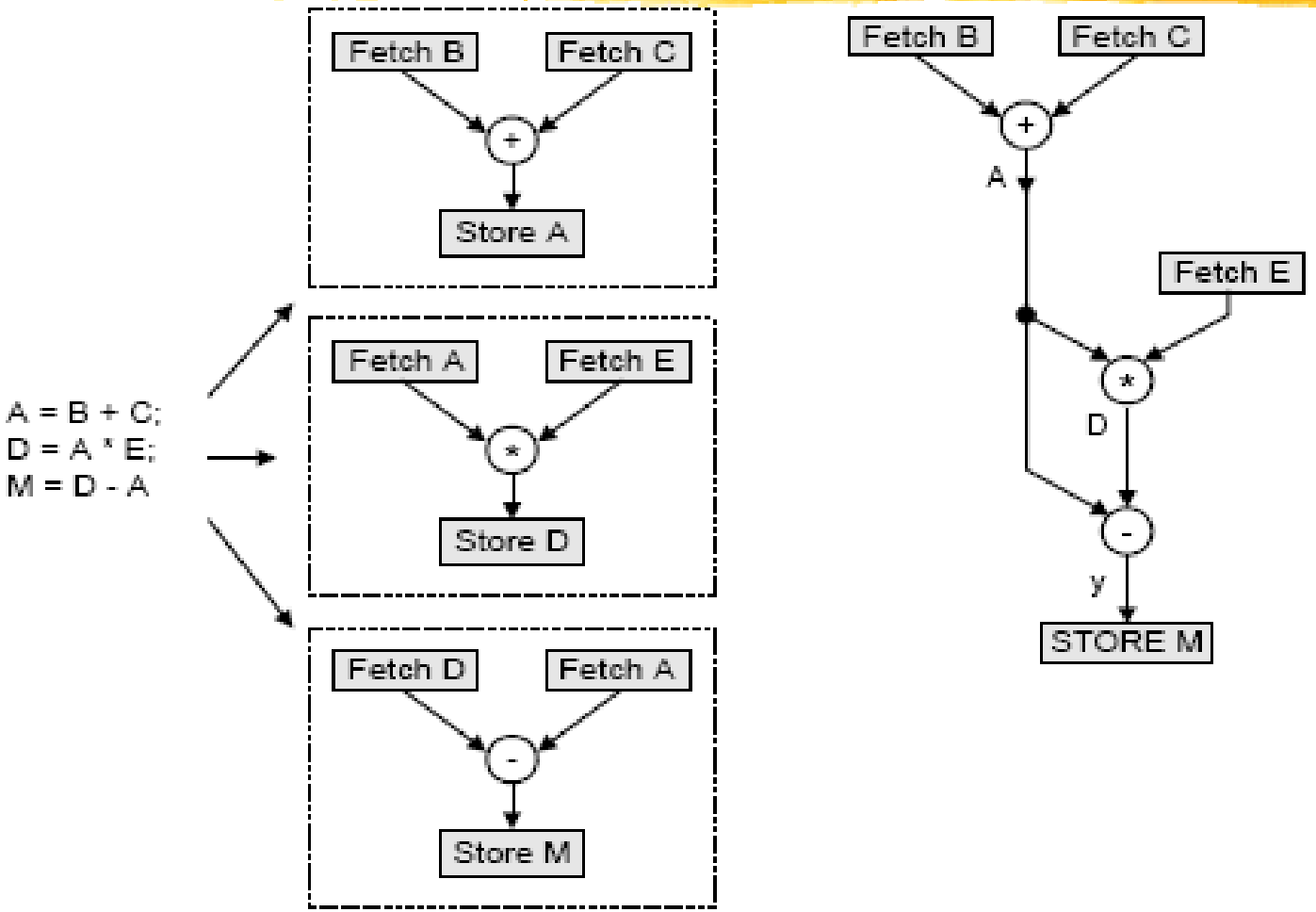
Behavior Synthesis



- Scheduling (assign operations to clock cycles)
- hardware resource allocation (to reduce overall hardware cost)
- FSM formulation (finite & deterministic)
- Memory inference (arrays to memory)
- Chaining (to reduce resource count)
- Pipelining (to improve clock cycle time)
- multi-cycle operations (schedule an operation to multiple cycles if the delay > T)
- Sequential re-timing (for performance & resource count, overlaps with sequential optimization)
- arithmetic transformation (ripple versus cla, array multiplier versus wallace tree etc)

BOSTON UNIVERSITY

Resource allocation and scheduling: Parse Trees and data Flow Graphs



Data dependencies

- Data dependencies describe relationships between operations:
  - x <= a + b; value of x depends on a, b
- High-level synthesis must preserve data dependencies.

Data flow graph

- Data flow graph (DFG) models data dependencies.
- Does not require that operations be performed in a particular order.
- Models operations in a basic block of a functional model—no conditionals.
- Requires single-assignment form.

Data flow graph construction

original code:

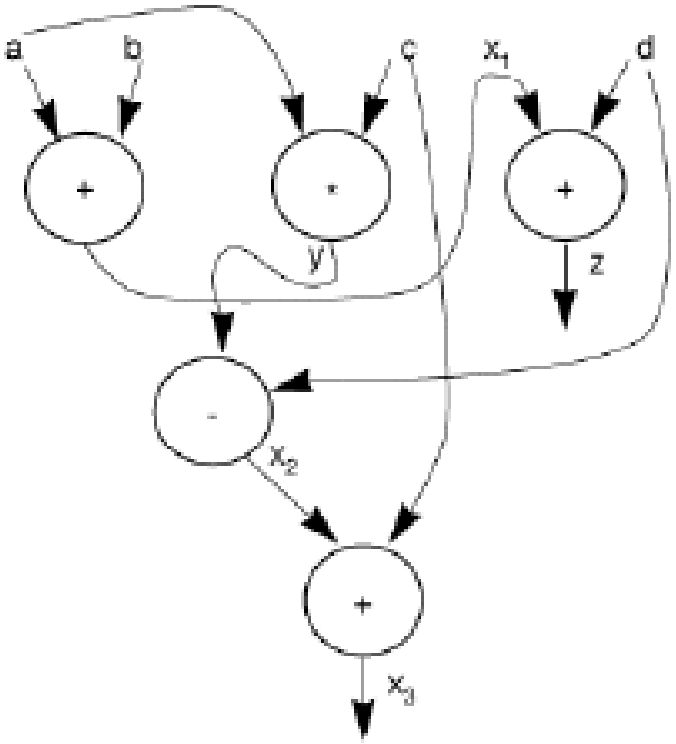
```
x <= a + b;
y <= a * c;
z <= x + d;
x <= y - d;
x <= x + c;
```

single-assignment form:

```
x1 <= a + b;
y <= a * c;
z <= x1 + d;
x2 <= y - d;
x3 <= x2 + c;
```

Data flow graph construction, cont'd

Data flow forms directed acyclic graph (DAG):

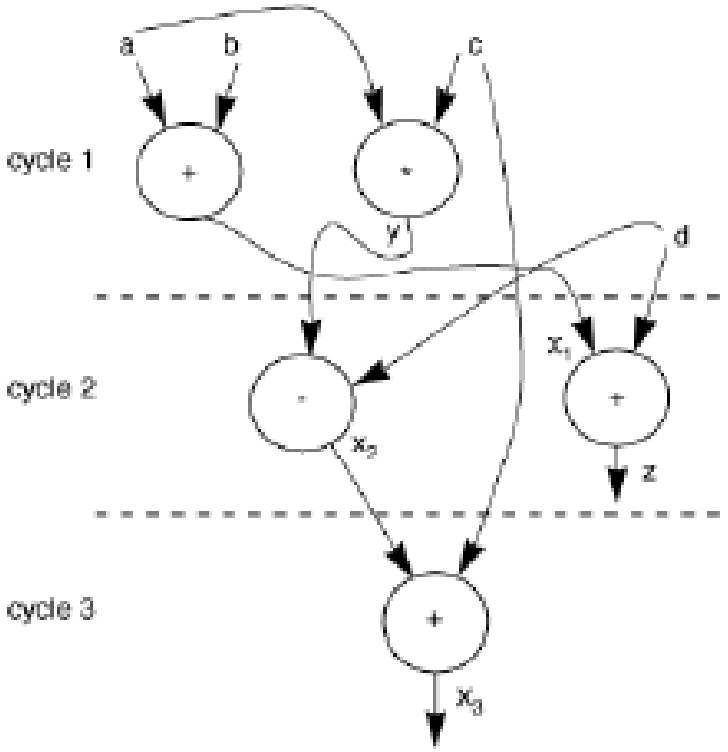


Goals of scheduling and allocation

- Preserve behavior—at end of execution, should have received all outputs, be in proper state (ignoring exact times of events).
- Utilize hardware efficiently.
- Obtain acceptable performance.

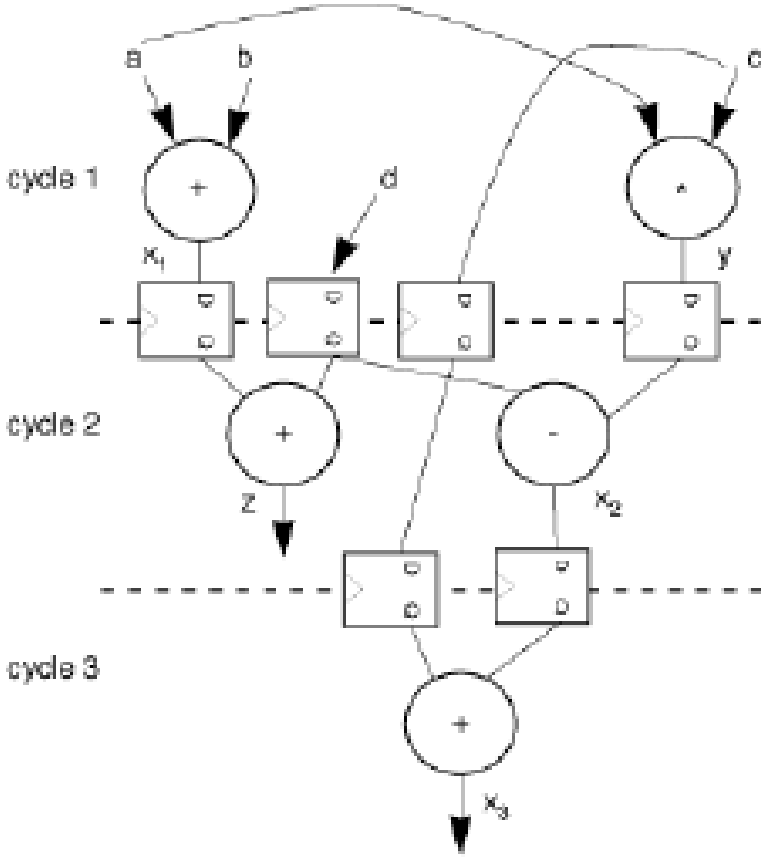
Data flow to data path-controller

One feasible schedule for last DFG:



Binding values to registers

registers fall on clock cycle boundaries

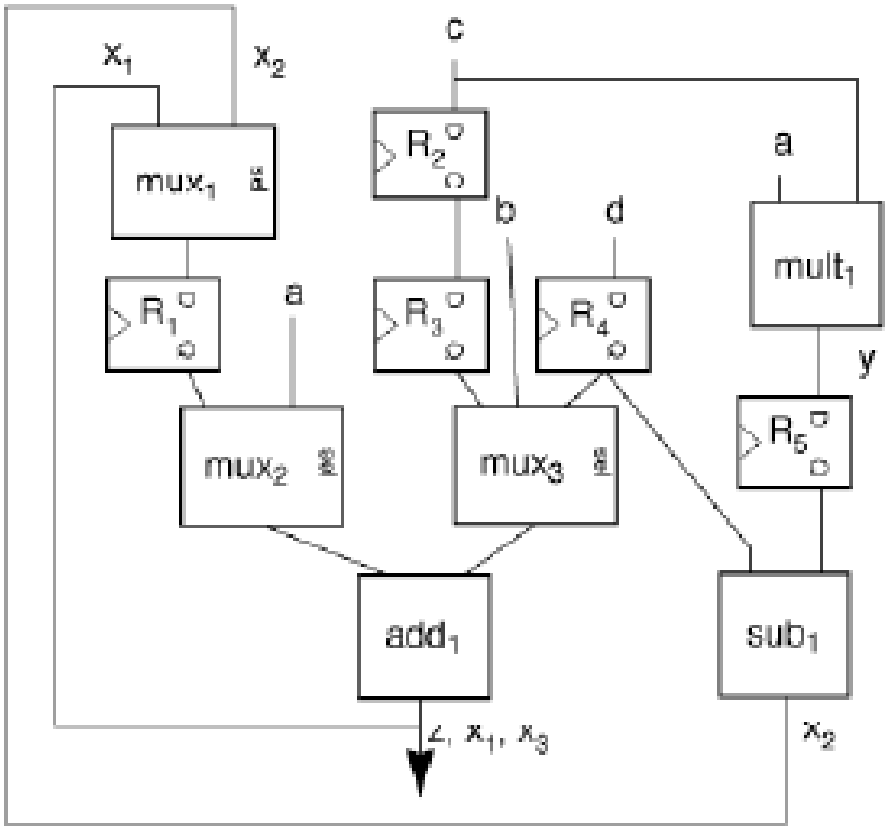


Allocation creates multiplexers

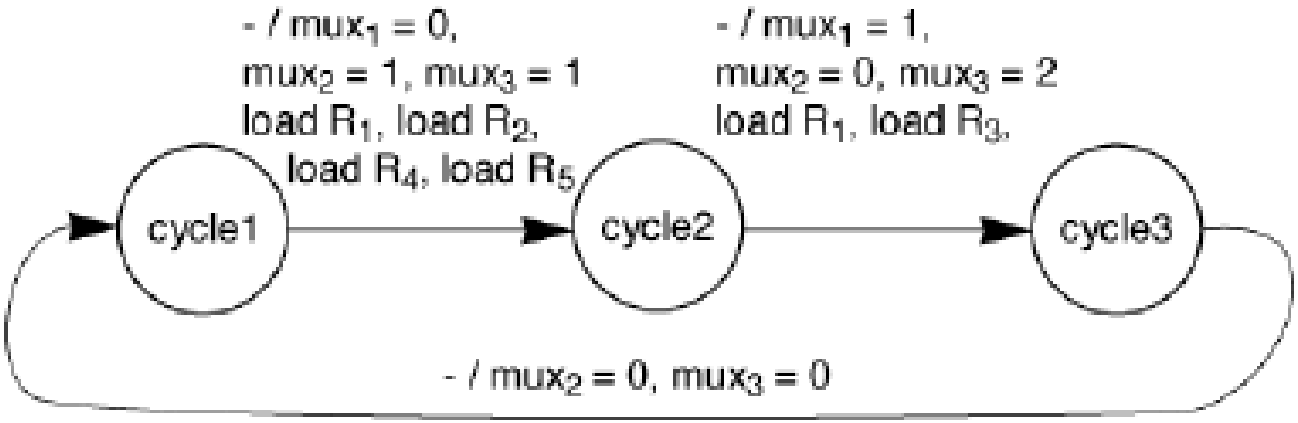
- Same unit used for different values at different times.
  - Function units.
  - Registers.
- Multiplexer controls which value has access to the unit.

Choosing function units

muxes allow function units to be shared for several operations



Building the sequencer



sequencer requires three states,
even with no conditionals

Choices during high-level synthesis

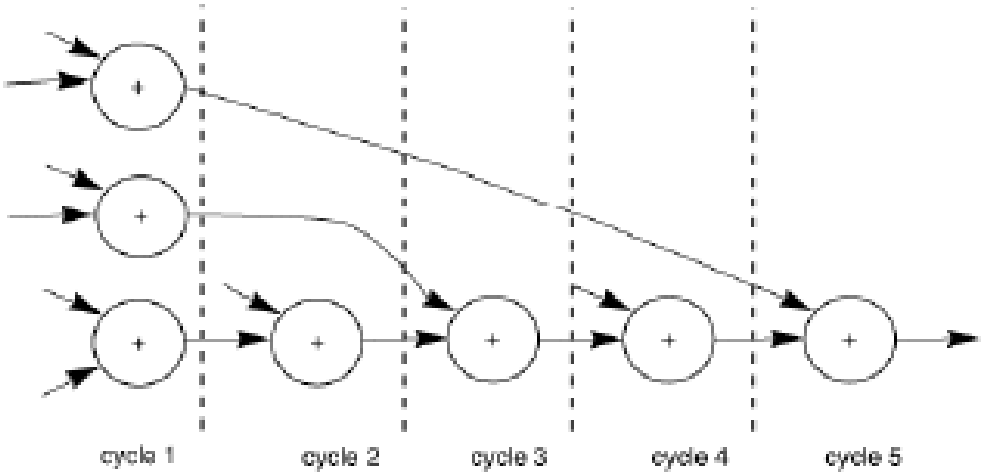
- Scheduling determines number of clock cycles required; binding determines area, cycle time.
- Area tradeoffs must consider shared function units vs. multiplexers, control.
- Delay tradeoffs must consider cycle time vs. number of cycles.

Finding schedules

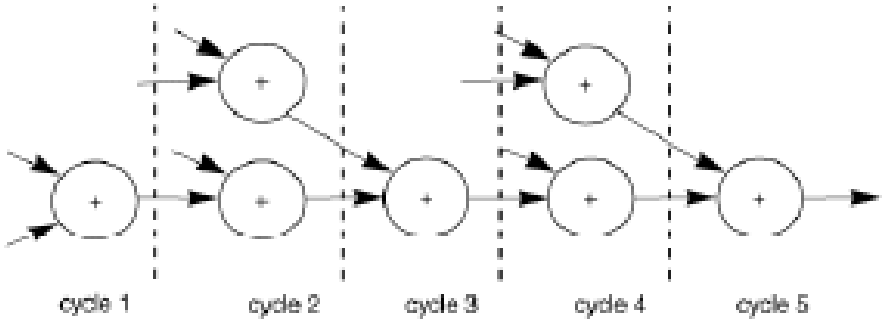
- Two simple schedules:
  - As-soon-as-possible (ASAP) schedule puts every operation as early in time as possible.
  - As-late-as-possible (ALAP) schedule puts every operation as late in schedule as possible.
- Many schedules exist between ALAP and ASAP extremes.

ASAP and ALAP schedules

ASAP

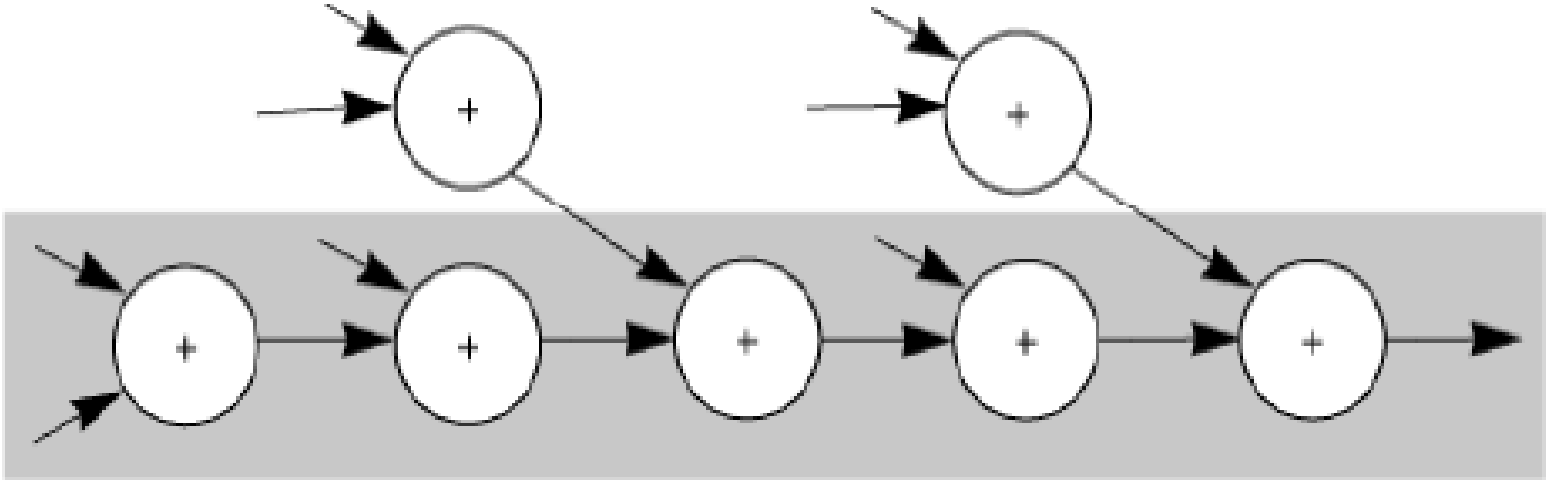


ALAP



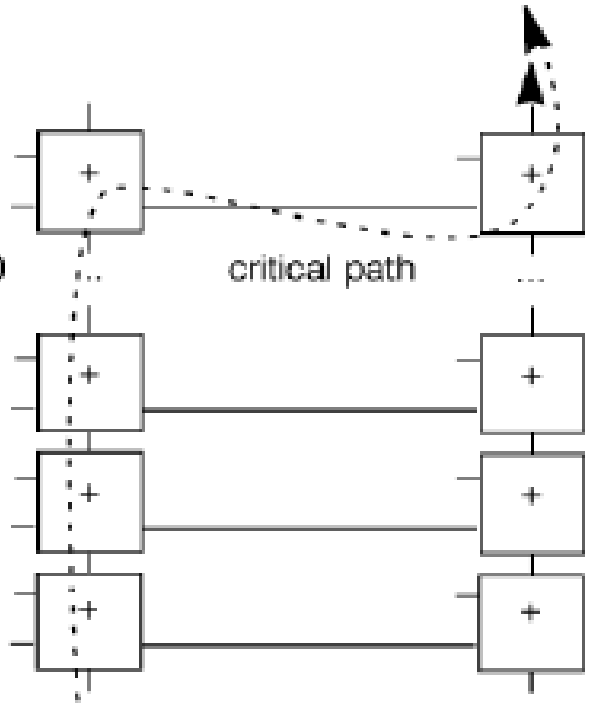
Critical path of schedule

Longest path through data flow determines minimum schedule length:



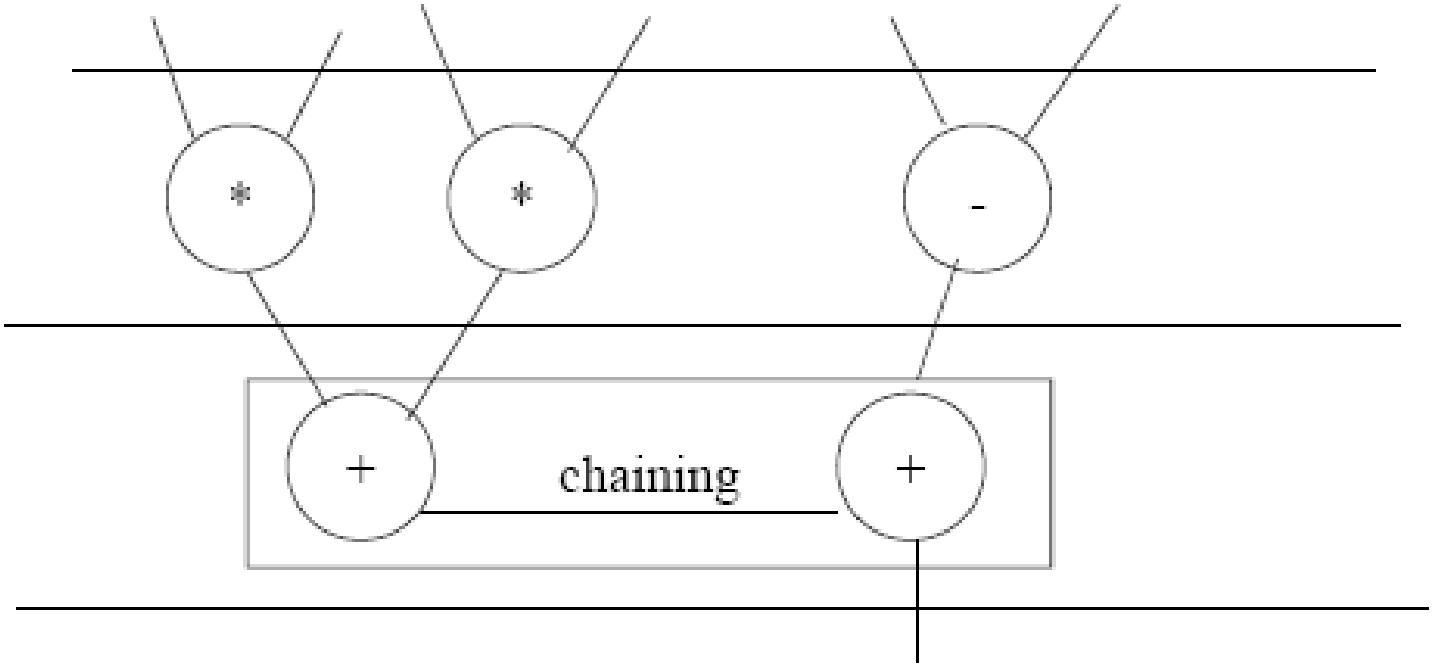
Operator chaining

- May execute several operations in sequence in one cycle—operator chaining.
- Delay through function units may no be additive, such as through several adders.



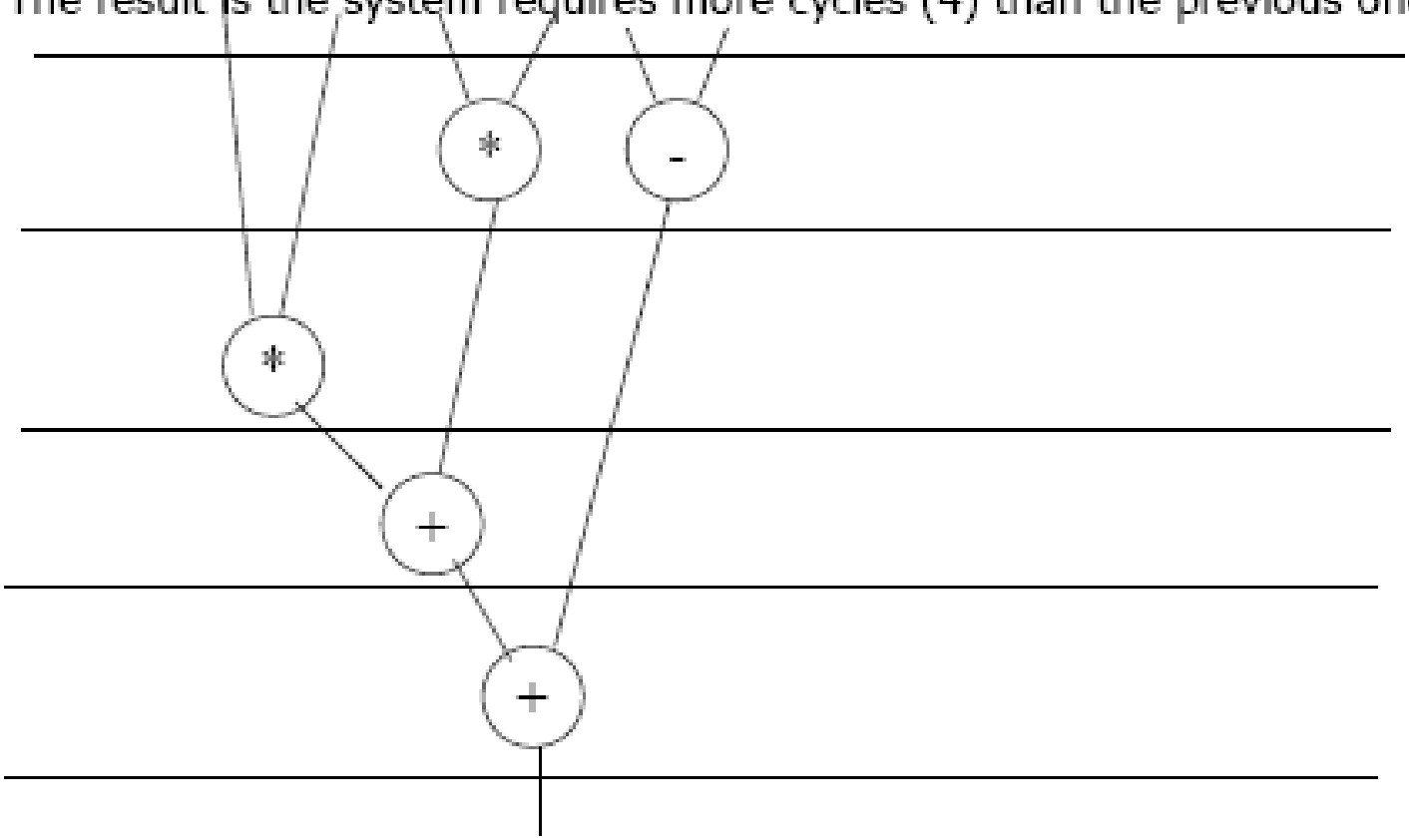
Schedule with throughput Constraints

- Uses more resource
- Use Chaining (the two adders)



Schedule with resource Constraints

- If the design is resource limited (e.g. two resources max in a cycle)
- The result is the system requires more cycles (4) than the previous one (2)



Control implementation

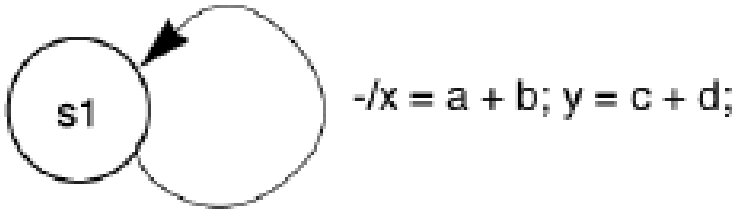
- Clock cycles are also known as control steps.
- Longer schedule means more states in controller.
- Cost of controller may be hard to judge from casual inspection of state transition graph.

Controllers and scheduling

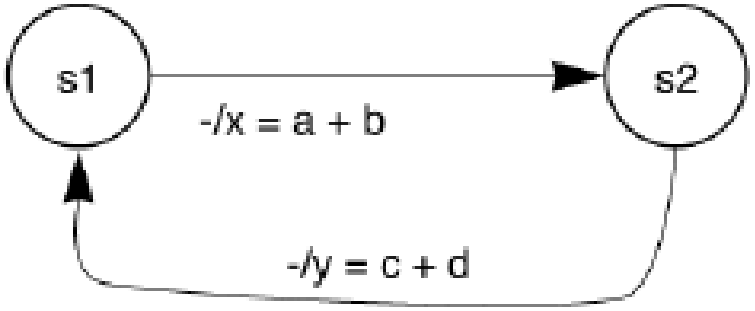
functional model:

x <= a + b;

y <= c + d;

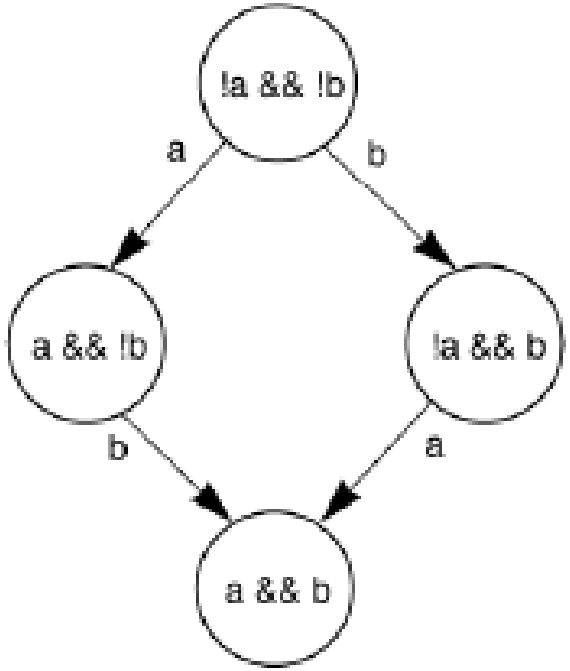


one state

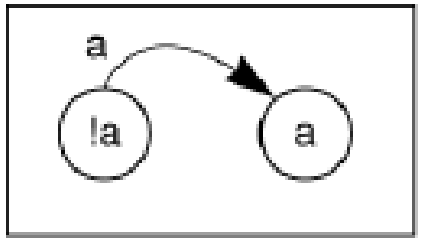


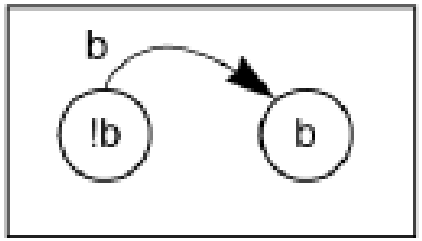
two states

Distributed control



one centralized controller

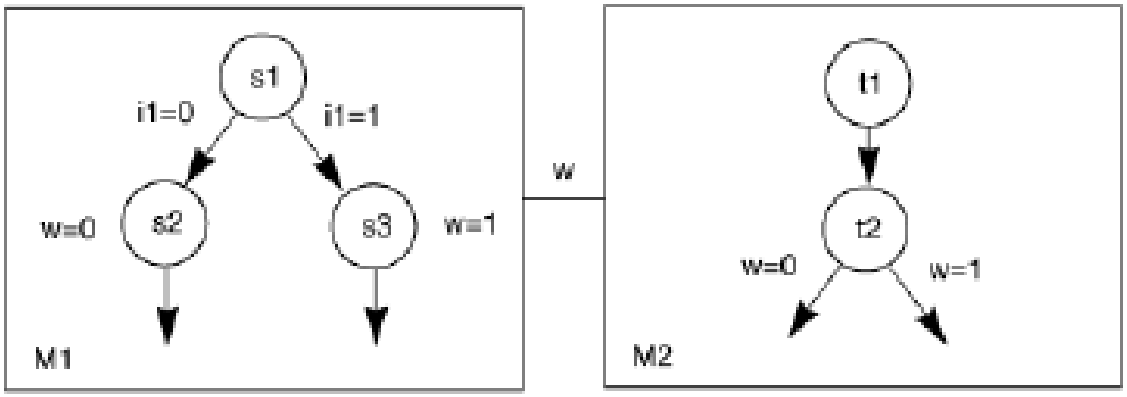




two distributed controllers

Synchronized communication between FSMs

To pass values between two machines, must schedule output of one machine to coincide with input expected by the other:

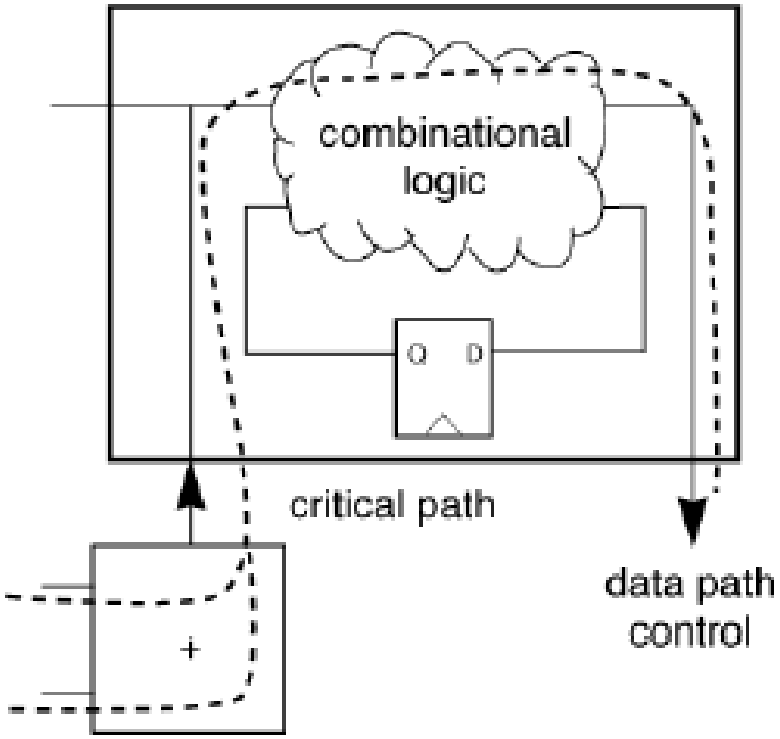


Hardwired vs. microcoded control

- Hardwired control has a state register and "random logic."
- A microcoded machine has a state register which points into a microcode memory.
- Styles are equivalent; choice depends on implementation considerations.

Data path-controller delay

Watch out for long delay paths created by combination of data path and controller:



History of Synthesis Technologies

- Synthesis technologies have been driven by market need. You can also get a good history of computers and internet at: http://www.webmythology.com
- VLSI design typically centers around the most popular computer/processors and the "word size" reflect design complexity during the time.

Time Line	Industry Design Complexity	Synthesis Algorithms Used	Logic Implementation Type	CPU/Processors that drive VLSI design during the time
Up to 1950	Simple switches and vacuum tubes	K-map	2-level	Simple control logic
Up to late 1970	4-bit data manipulation	Quine-McCluskey	2-level	Intel 4004/4bit/1971, Rockwell PPS-4/4bit/1972, National IMP-16/4bit/1973, PAL22V10
Up to early 1980	8-bit data manipulation	Heuristic two-level Methods	2-level	Fairchild F8/8bit/1975, PDP-11/16bit/1970
Since early 1990	16/32-bit data manipulation and beyond	**Multiple Level Logic Algorithms**	Multiple level	DEC/VAX, Sun/Sparc, MIPS and other popular 32 bits processors

BOSTON UNIVERSITY

Logic Optimization: History

Presently, there appears to be a limit of ~20-25 input variables in problems that can be handled by exact minimizers

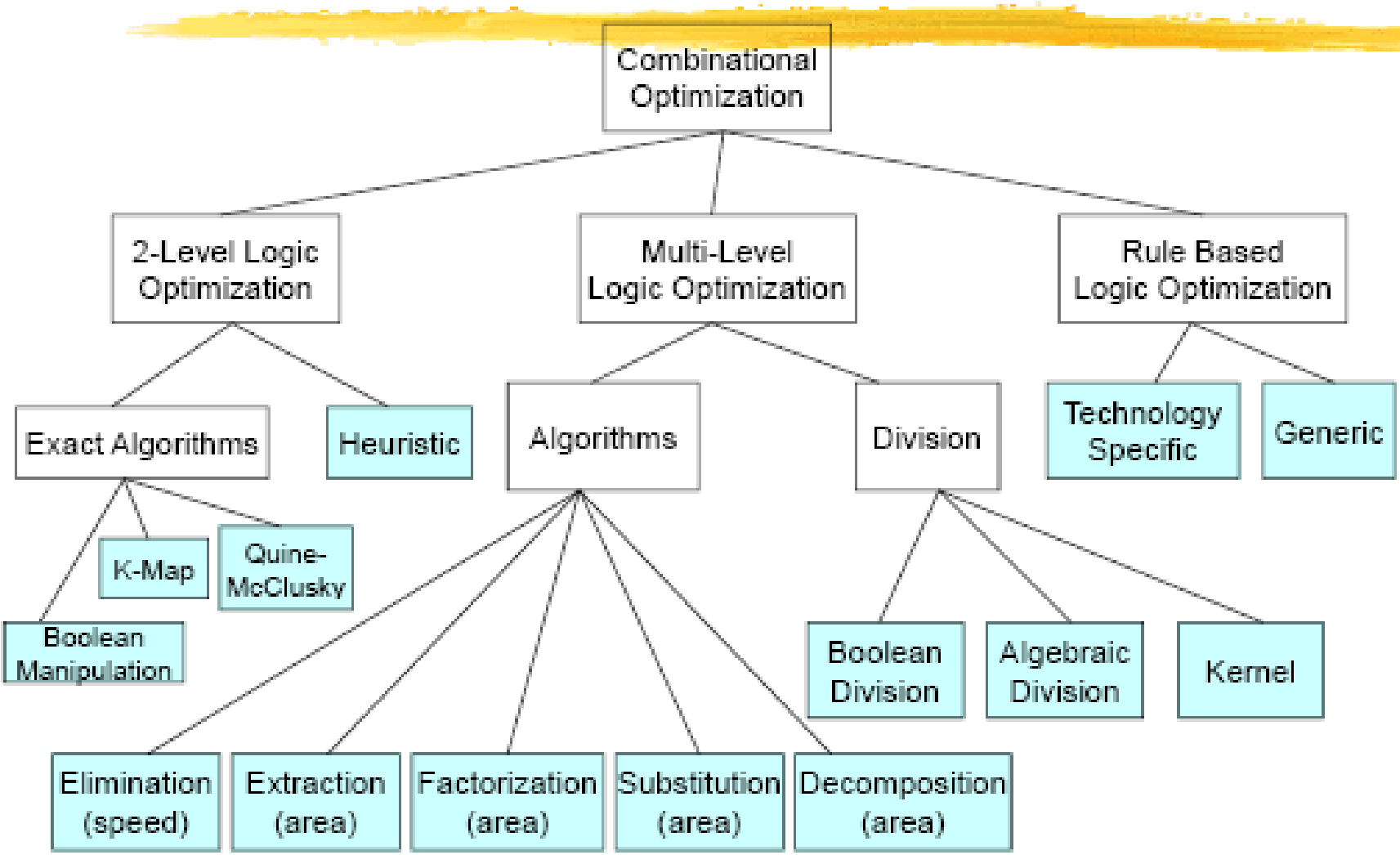
Easy for complex control logic to exceed 20- 25 input variables

HISTORY

50's	Karnaugh Map	≤ 5 variables
60's	Q-M method	< 10 variables
70's	Starner, Dietmeyer	< 15 variables
1974	MINI	heuristic
1980-84	ESPRESSO	approaches
1986	McBoole	< 25 variables
1987	ESPRESSO-EXACT	< 25 variables

BOSTON UNIVERSITY

Combinational Synthesis



BOSTON UNIVERSITY

Syntax-directed translation

- Translate HDL into logic directly.
  - ab + ac
- Generally requires optimization.

BOSTON UNIVERSITY

Macros

- Pre-designed logic.
  - Generally identified by language features.
  - + operator.
  - xxx()
- Hard macro: includes placement.
- Soft macro: no placement.

BOSTON UNIVERSITY

Logic synthesis phases

- Technology-independent optimizations work on logic representations that do not directly model logic gates.
- Technology-dependent optimizations work in the available set of logic gates.
- Transformation from technology-independent to technology-dependent is called library binding.

BOSTON UNIVERSITY

Technology-independent optimizations

- Works on Boolean expression equivalent.
- Estimates size based on number of literals.
- Uses factorization, resubstitution, minimization, etc. to optimize logic.
- Technology-independent phase uses simple delay models.

BOSTON UNIVERSITY

Technology-dependent optimizations

- Maps Boolean expressions into a particular cell library.
- Mapping may take into account area, delay.
- May perform some optimizations on addition to simple mapping.
- Allows more accurate delay models.

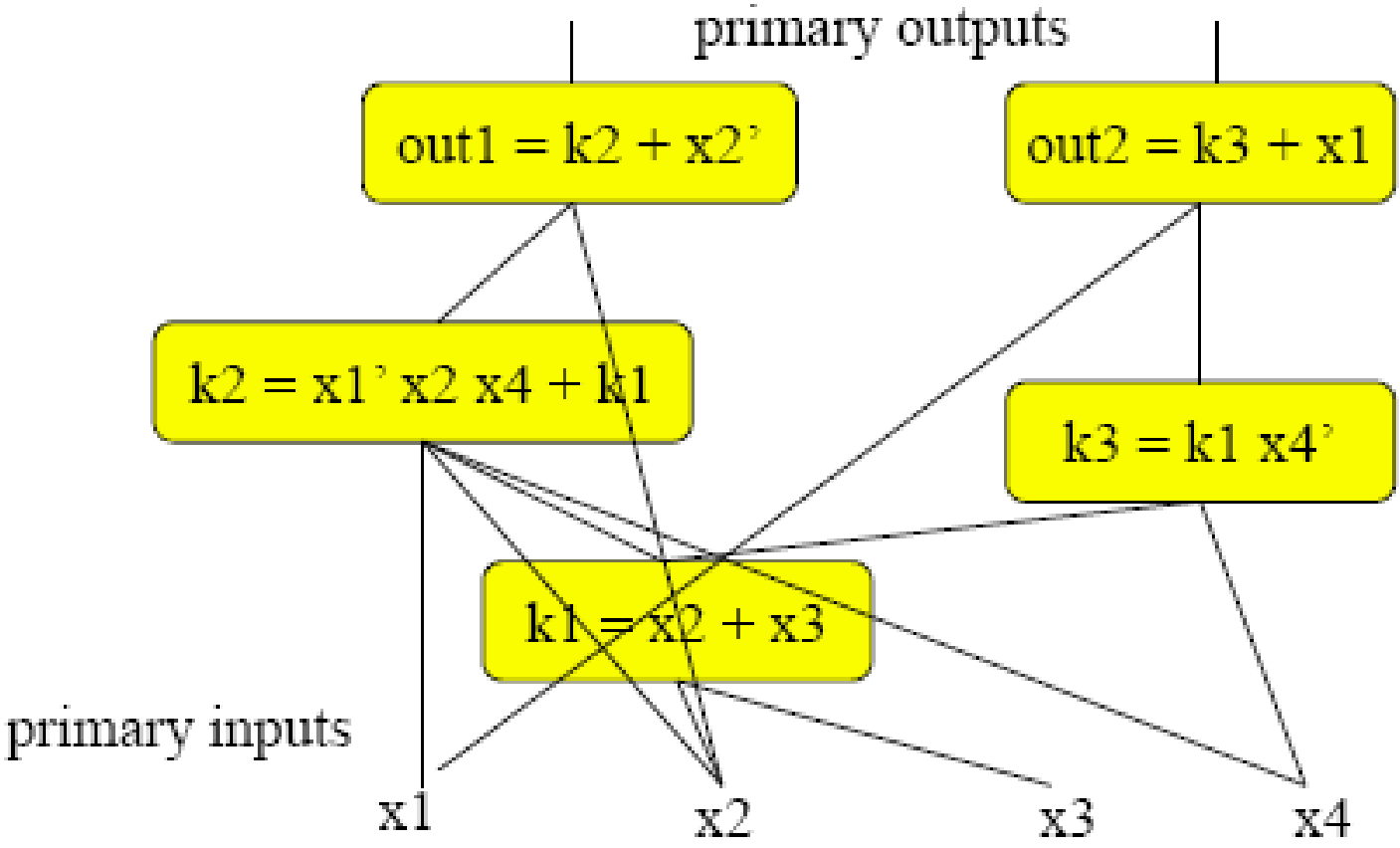
BOSTON UNIVERSITY

Boolean network

- A Boolean network is the main representation of the logic functions for technology independent optimizations.
- Each node can be represented as sum-of-products (or product-of-sums).
- Provides multi-level structure, but functions in the network need not correspond to logic gates.

BOSTON UNIVERSITY

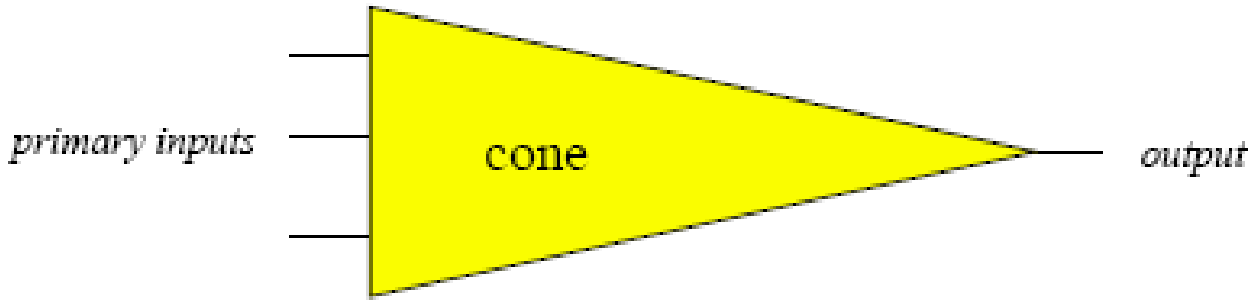
Boolean network example



BOSTON UNIVERSITY

Terms

- Support: set of variables used by a function.
- Transitive fanout: all the primary outputs and intermediate variables of a function.
- Transitive fanin: all the primary inputs and intermediate variables used by a function. Transistive fanin determines a cone of logic.



BOSTON UNIVERSITY

Cost in the Boolean network

- Don't know exact gate structure, but can estimate final network cost:
  - area estimated by number of literals (true or complement forms of variables);
  - delay estimated by path length.

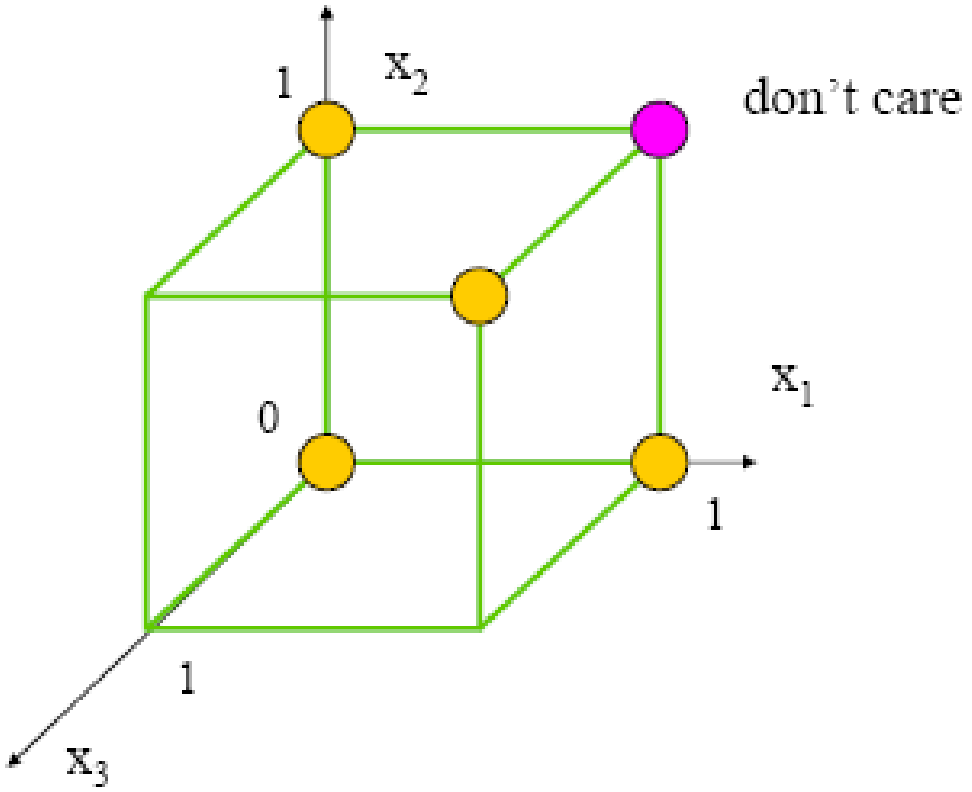
BOSTON UNIVERSITY

Partially-specified functions

- Don't-cares can be implemented in either the on-set or off-set.
- Don't-cares provide the greatest opportunities for minimization in many cases.

BOSTON UNIVERSITY

Partially-specified function example



BOSTON UNIVERSITY

Don't-cares in Boolean networks

- In two-level function, don't-cares are defined at primary output.
- In Boolean network, structure of network itself introduces don't-cares.

BOSTON UNIVERSITY

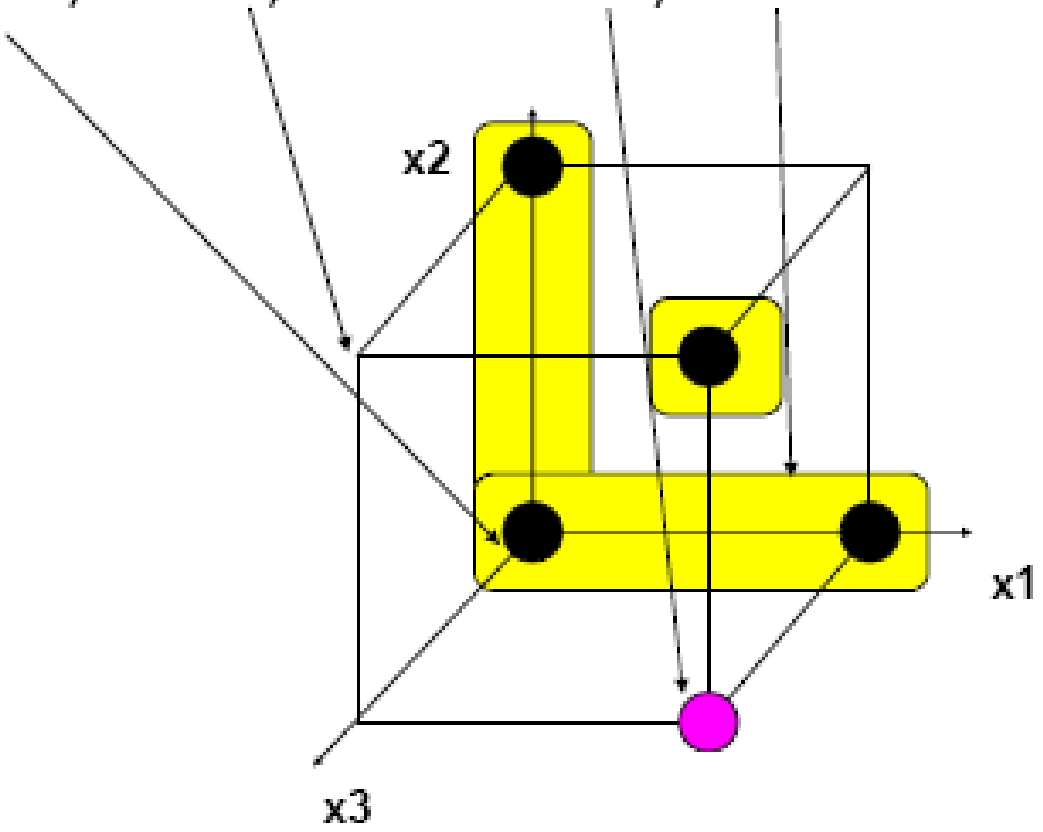
Optimizations

- Simplification.
  - Changing the way a function is represented.
- Network restructuring.
  - Adding and removing nodes.
- Delay restructuring.
  - Optimizations that reduce the height of critical paths.

BOSTON UNIVERSITY

Cube representation

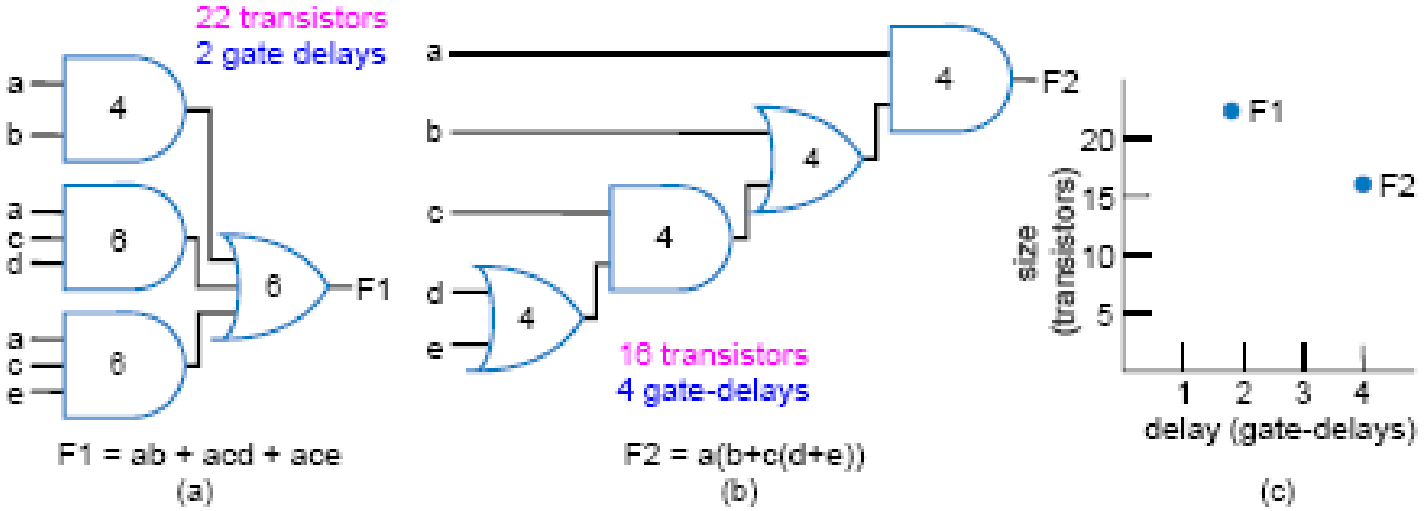
- On-set, off-set, don't-care set, cover:



BOSTON UNIVERSITY

Multi-Level Logic Optimization –
Performance/Size Tradeoffs

- We don't always need two level logic
  - Multiple levels may yield fewer gates
  - Example
    - F1 = ab + acd + ace → F2 = ab + ac(d + e) = a(b + c(d + e))
    - General technique: Factor out literals – xy + xz = x(y+z)

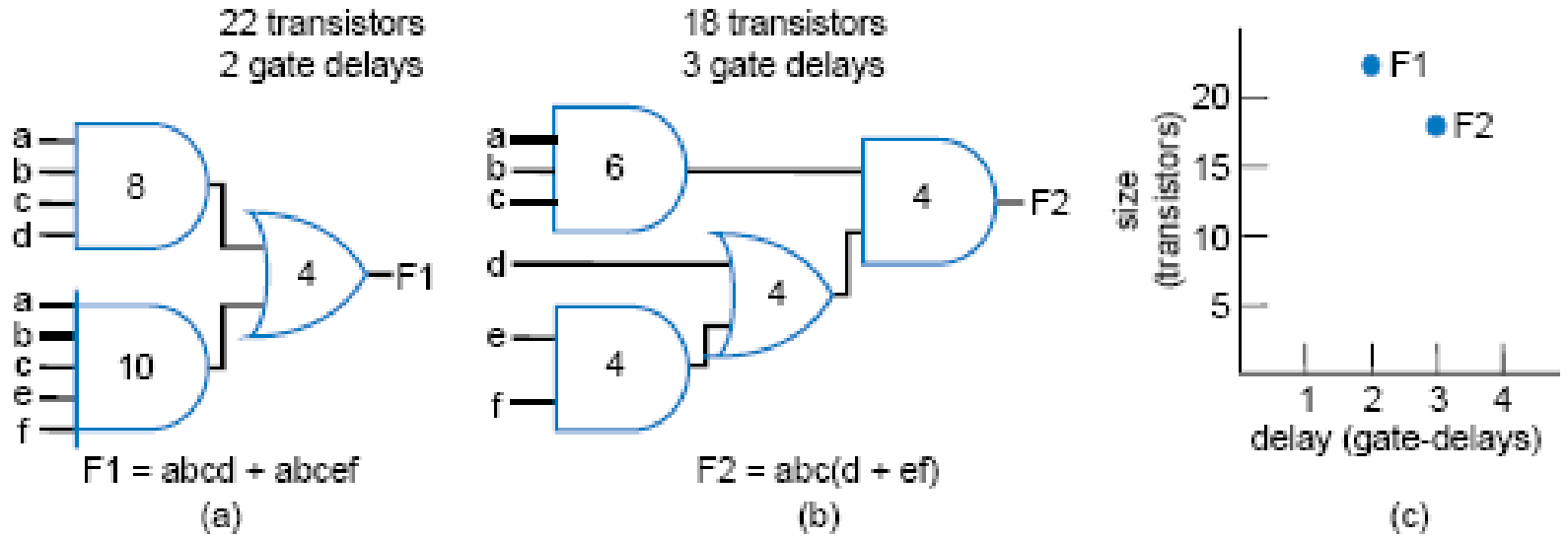


BOSTON UNIVERSITY

Multi-Level Example

- Q: Use multiple levels to reduce number of transistors for
  - F1 = abcd + abcef

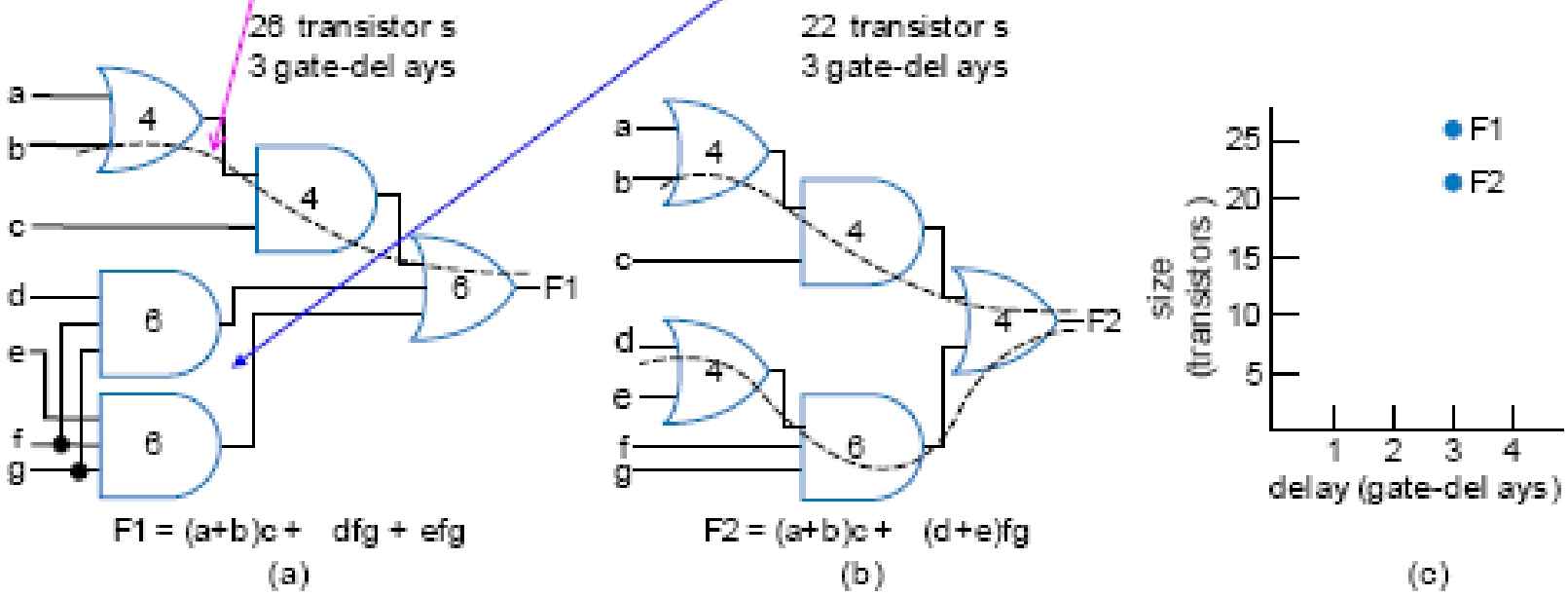
- A: abcd + abcef = abc(d + ef)
  - Has fewer gate inputs, thus fewer transistors



BOSTON UNIVERSITY

Multi-Level Example: Non-Critical Path

- Critical path: longest delay path to output
- Optimization: reduce size of logic on non-critical paths by using multiple levels



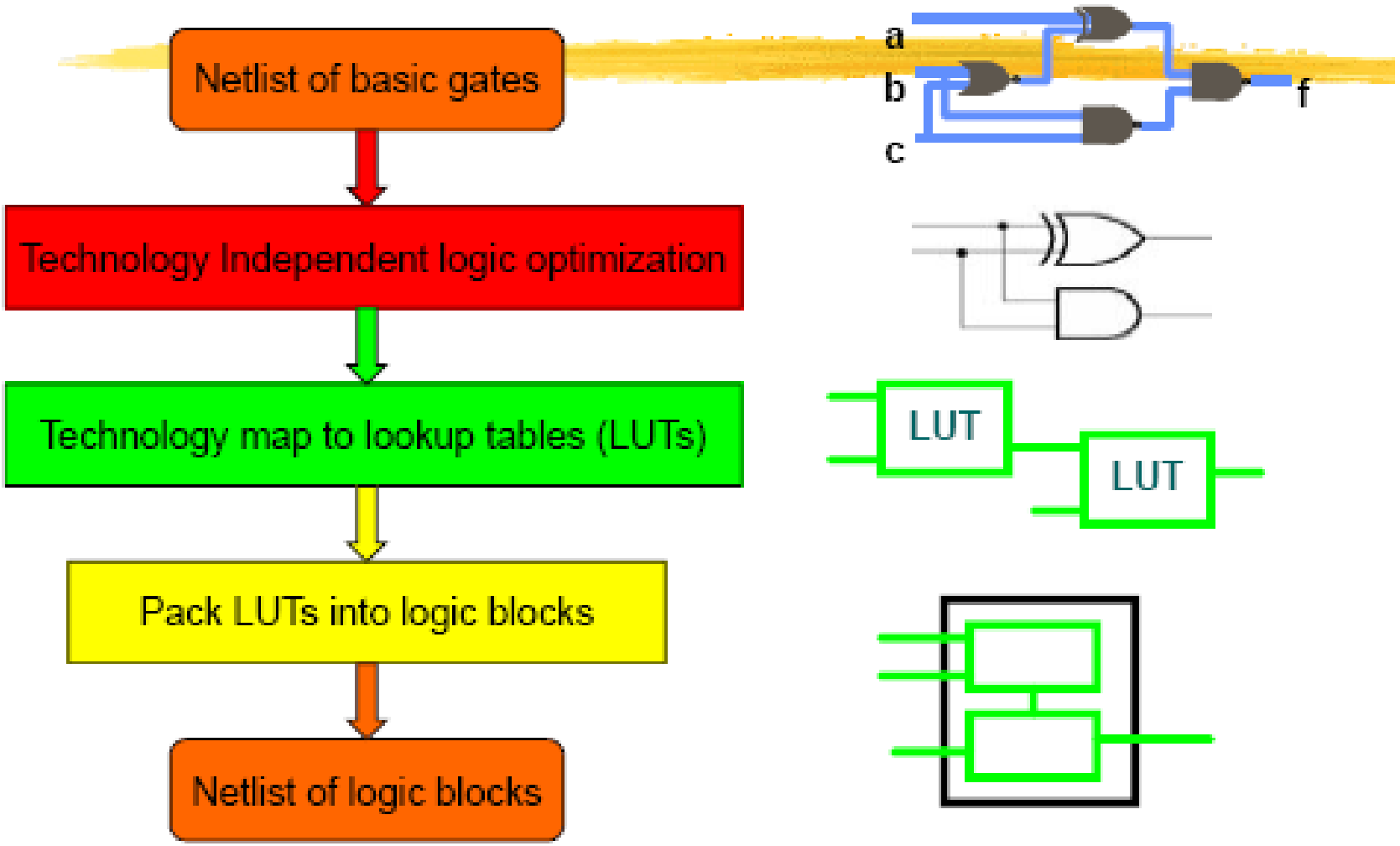
BOSTON UNIVERSITY

Automated Multi-Level Methods

- Main techniques use heuristic iterative methods
  - Define various operations
    - "Factor out": xy + xz = x(y+z)
    - Expand, and others
  - Randomly apply, see if improves
    - May even accept changes that worsen, in hopes eventually leads to even better equation
    - Keep trying until can't find further improvement
  - Not guaranteed to find best circuit, but rather a good one

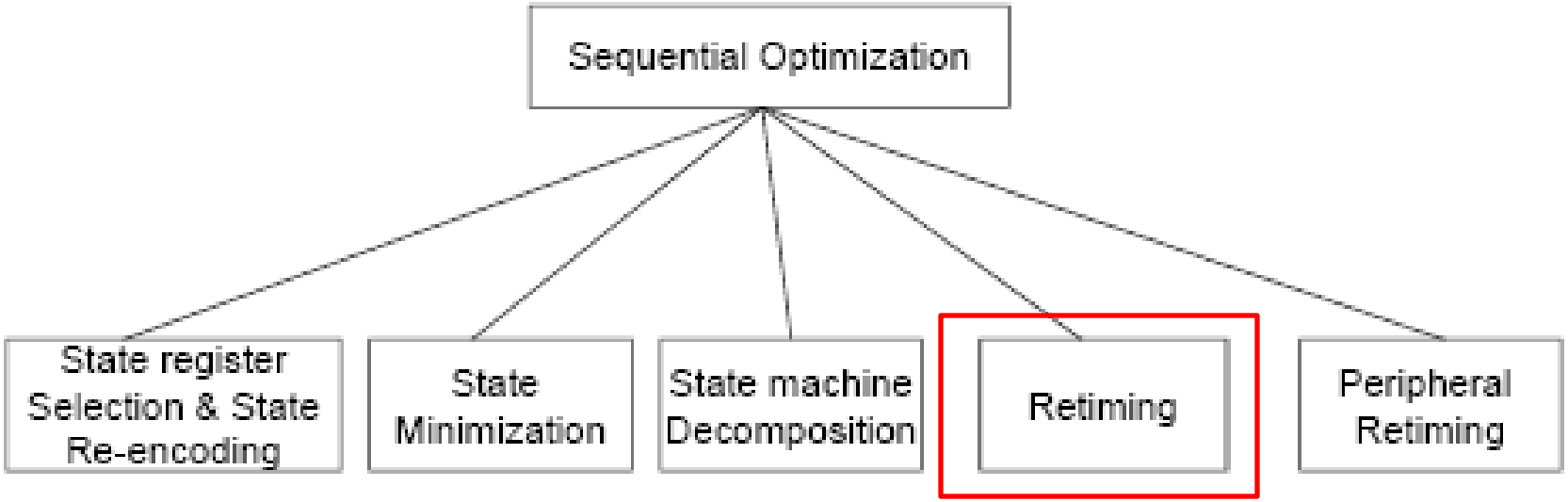
BOSTON UNIVERSITY

Logic Optimization



BOSTON UNIVERSITY

Sequential Synthesis



- Sequential synthesis is not widely used in ASIC design **Discussed in future lecture**
- Some of the issues with sequential synthesis include:
  - Changes in timing boundaries (making debugging more difficult)
  - Makes it difficult for subsequent formal verification process

BOSTON UNIVERSITY

Next Time

- Abstract State Machines

Questions?

BOSTON UNIVERSITY